# Indicador de Sentimiento Económico (ISE): simulador cliente/servidor con procesos, hilos y pipes nominales en C

Daniel Plazas, Juan Camilo Maldonado y Samuel Zeudec

Facultad de Ingeniería, Pontificia Universidad Javeriana

Sistemas Operativos 2026-30

John Jairo Corredor Franco

4 de noviembre de 2026

Pontificia Universidad Javeriana

## Introducción

El Indicador de Sentimiento Económico (ISE) es un número que resume que tan optimistas o pesimistas están las empresas y los consumidores. Se calcula como un promedio ponderado de encuestas de cinco sectores: industria (40 %), servicios (30 %), consumidores (20 %), comercio minorista (5 %) y construcción (5 %). En este proyecto construimos un simulador de ese indicador para Linux, escrito en C y apoyado en llamadas al sistema POSIX. Con el resultado, extraemos una recomendación de inversión: vender si el ISE supera 50, comprar si es menor que 50 y mantenerse neutro si es igual a 50.

La solución se compone de tres programas: un generador de encuestas (*generadorCSV*), que crea un fichero CSV por país; uno o varios agentes de lectura (*agentePais*), que envían esas encuestas; y un servidor (*servidorISE*), que las recibe, calcula los resultados y presenta el resumen final. Los agentes y el servidor se comunican por un pipe nominal. Este documento explica los conceptos de sistemas operativos en que se apoya la solución, la metodología seguida, el plan de pruebas con los tiempos medidos, su análisis y las conclusiones.

### Objetivos

**Objetivo general:** Desarrollar un simulador del ISE con arquitectura cliente/servidor en C, que use procesos e hilos POSIX, mecanismos de sincronización y comunicación entre procesos mediante pipes nominales.

**Objetivos específicos:**

a) Resolver el problema con procesos e hilos de la biblioteca POSIX.

b) Sincronizar hilos y procesos con semáforos y *mutex* sin condiciones de carrera ni interbloqueos.

c) Comunicar agentes y servidor por un pipe nominal sin que se mezclen los mensajes.
d) Validar las llamadas al sistema y liberar todos los recursos.
e) Medir el tiempo que toma el sistema al variar la cantidad de encuestas, el tamaño del buffer y la cantidad de agentes.

**Conceptos fundamentales**

**Proceso:** Un proceso es un programa en ejecución; además del código incluye su contador de programa, su pila y sus datos (Silberschatz et al., 2018). En POSIX, la llamada *fork* crea un proceso hijo que es una copia del padre, y *waitpid* permite al padre esperar a que el hijo termine (Kerrisk, 2010). El servidor ISE usa esta pareja: el padre crea y vigila, y el hijo hace el trabajo.

**Hilo:** Un hilo es la unidad básica de uso de la CPU dentro de un proceso. Los hilos de un mismo proceso comparten el código, los datos y los archivos abiertos, pero cada uno tiene su propia pila y su propio contador de programa (Silberschatz et al., 2018). La biblioteca POSIX de hilos (*pthread*) ofrece funciones como *pthread_create* y *pthread_join*. Si un proceso con varios hilos llama a *fork*, el hijo conserva solamente el hilo que hizo la llamada (Kerrisk, 2010); por eso el servidor crea sus hilos después del *fork*.

**Pipe nominal (FIFO):** Un pipe es un canal de comunicación unidireccional entre procesos: lo que escribe un extremo lo lee el otro en el mismo orden. El pipe nominal o FIFO tiene un nombre en el sistema de archivos, de modo que puede ser usado por procesos que no son parientes entre sí, y se crea con *mkfifo* (Kerrisk, 2010). En un pipe, una escritura de no más de *PIPE_BUF* bytes es atómica, es decir, no se mezcla con las escrituras de otros procesos (IEEE y

The Open Group, 2018). El pipe tiene además una capacidad limitada, por lo que quien escribe espera cuando se llena (Kerrisk, s. f.).

**Condición de carrera y sección crítica:** Una condición de carrera ocurre cuando varios hilos acceden a la vez a datos compartidos y el resultado depende del orden en que se ejecuten. La parte del código que accede a esos datos es la sección crítica y debe ejecutarse con exclusión mutua (Silberschatz et al., 2018).

**Mutex:** Un *mutex* es un candado de exclusión mutua: solo el hilo que lo tiene bloqueado puede estar dentro de la sección crítica (*pthread_mutex_lock* y *pthread_mutex_unlock*) (Silberschatz et al., 2018).

**Semáforo:** Un semáforo es una variable entera que solo se modifica con dos operaciones atómicas: esperar (*sem_wait*), que decrementa el valor y bloquea al hilo si no hay unidades disponibles, y señalar (*sem_post*), que lo incrementa y puede despertar a otro hilo (Silberschatz et al., 2018). La variante *sem_timedwait* agrega un tiempo máximo de espera.

**Productor-consumidor y buffer circular:** En el problema del productor y el consumidor, un hilo produce datos y otro los consume a través de un buffer de tamaño limitado; dos semáforos evitan que el productor escriba en un buffer lleno y que el consumidor lea de uno vacío. En un buffer circular, los índices de escritura y de lectura vuelven al inicio al llegar al final (Silberschatz et al., 2018).

**Interbloqueo:** Un interbloqueo (*deadlock*) ocurre cuando varios hilos esperan para siempre recursos que los otros no liberan (Silberschatz et al., 2018). Para evitar que un hilo quede esperando indefinidamente, el servidor usa esperas con tiempo límite.

**Rendimiento:** Para medir el desempeño se usan el tiempo de ejecución y el rendimiento (*throughput*), que es la cantidad de trabajo completado por unidad de tiempo (Silberschatz et al.,

2018). En este proyecto el trabajo es una encuesta y el rendimiento se expresa en encuestas por segundo.

**Descripción del problema**

El generador recibe la sigla del país (ES, FR o IT) y un porcentaje entre 1 y 100, y crea el fichero *encuestas_XX.csv* con una cantidad de encuestas igual al máximo de la Tabla 1 por sector multiplicado por el porcentaje, redondeada al entero más cercano. El agente lee ese fichero, se conecta al servidor, envía las encuestas, muestra el progreso cada 500 envíos y devuelve el control a la consola. El servidor atiende hasta cuatro agentes, emite un reporte parcial cuando cada uno termina y un resumen general cuando se cumple el tiempo indicado.

**Tabla 1**. *Máximo de encuestas por sector y por país*

| País | Industria | Servicios | Consumidores | Comercio | Construcción | Total |
|---|---|---|---|---|---|---|
| ES | 1760 | 1430 | 2020 | 1080 | 290 | 6580 |
| FR | 3850 | 4240 | 1670 | 2860 | 24020 | 36640 |
| IT | 5440 | 3100 | 2000 | 1560 | 1000 | 13100 |

Los tres programas se invocan con banderas que pueden escribirse en cualquier orden:

```
$ ./servidorISE -b <tamB> -p <pipe> -t <segundos> -a <agentes>
$ ./agentePais  -n <encuestas_XX.csv> -p <pipe>
$ ./generadorCSV -p <ES|FR|IT> -e <1-100>
```

La bandera -b es el tamaño del buffer en registros por agente, -t el tiempo de operación del servidor (entre 30 y 120 segundos) y -a la cantidad máxima de agentes (entre 1 y 4). Cualquier valor fuera de rango termina el programa con un mensaje de error y código de salida 1.

## Metodología

El proyecto se desarrolló siguiendo los pasos que se describen a continuación, desde el análisis del enunciado hasta la validación del sistema integrado.

1. **Análisis del enunciado y reparto de roles.** A partir del enunciado se identificaron los tres programas y se asignó una persona por componente (Tabla 3). Para que los tres avanzaran en paralelo, *servidorISE.h* se tomó como contrato común: define el mensaje del pipe y las tres funciones públicas del servidor (*servidor_parsear_argumentos*, *servidor_validar_argumentos* y *servidor_ejecutar*), de modo que el agente se escribió contra esos nombres sin depender de la implementación del servidor.
2. **Diseño del protocolo de mensajes.** En *servidorISE.h* se definieron la enumeración *TipoMensaje* y la estructura *MensajeEncuesta* (tipo, país de hasta tres caracteres, nombre del sector, su identificador y el sentimiento). Todos los campos tienen tamaño fijo, así que cada mensaje ocupa 40 bytes y el servidor siempre lee un bloque completo con *read*. Como 40 bytes es mucho menos que *PIPE_BUF*, cada *write* es atómica; el agente lo verifica al compilar con *_Static_assert* y pone la estructura en cero con *memset* antes de llenarla (*enviar_mensaje*, en *agentePais.c*).
3. **Implementación del generador.** En *generadorCSV.c*, *getopt* lee *-p* y *-e* en cualquier orden. *buscar_pais* valida la sigla y devuelve su posición en la Tabla 1, que es también el *ID_Pais*, y *leer_porcentaje* usa *strtol* para aceptar solo enteros de 1 a 100. El *main* calcula la cantidad por sector con (máximo * porcentaje + 50) / 100, donde el +50 redondea al entero más cercano. *escribir_sector* escribe cada fila con un identificador consecutivo y un sentimiento de *sentimiento_aleatorio*, que escala *rand()* a 0-100. La semilla mezcla la hora y el PID, y si una escritura falla se borra el fichero para no dejar un CSV incompleto.

4. **Implementación del servidor.** En *servidorISE.c*, *servidor_ejecutar* es el proceso padre: crea el pipe con *mkfifo*, crea al hijo con *fork*, espera con *waitpid*, borra el pipe con *unlink* e informa éxito o error, de modo que el pipe se elimina aunque el hijo falle. El hijo (*servidor_hijo*) prepara los buffers y semáforos (*inicializar_slots*), crea con *pthread_create* el receptor, el temporizador y un consumidor por agente, los espera con *pthread_join* y llama a *imprimir_resumen_general*. Las banderas se validan en *servidor_validar_argumentos* (*-t* de 30 a 120, *-a* de 1 a 4, *-b* mayor que cero), y *main.c* solo llama a las tres funciones públicas.
5. **Diseño de la sincronización.** Cada agente tiene una casilla con un buffer circular de *-b* registros y dos semáforos: *espacios_libres*, iniciado en *-b*, e *items_disponibles*, iniciado en 0. El receptor espera *espacios_libres*, escribe y señala *items_disponibles*; el consumidor hace lo inverso (*hilo_receptor* e *hilo_consumidor*). Con un único productor y un único consumidor por buffer, los índices no necesitan mutex. Las esperas pasan por *esperar_con_limite*, que usa *sem_timedwait* de un segundo para que ningún hilo quede bloqueado al cerrar. *mutex_asignacion* evita que dos países reciban la misma casilla, y *mutex_tiempo* protege la bandera de cierre, que activa *marcar_cierre* y consultan los demás con *hay_que_cerrar*. El receptor abre el pipe con *O_RDWR* y usa *poll* de 500 ms para revisar el cierre.
6. **Implementación del agente.** En *agentePais.c*, *parsear_argumentos* lee *-n* y *-p* con *getopt*. *leer_encuestas* recorre el CSV dos veces: la primera cuenta las líneas para reservar la memoria exacta y la segunda las valida (encabezado, seis columnas, sector de 0 a 4, sentimiento de 0 a 100) y descarta las dañadas. *abrir_pipe* comprueba con *stat* que sea un pipe nominal y lo abre solo para escritura y sin bloqueo; si no hay

servidor, *open* falla con *ENXIO* y el agente termina con un error claro. Después quita el modo sin bloqueo para que *write* espere si el pipe se llena. *enviar_encuestas* manda la conexión, una encuesta por mensaje con progreso cada 500 y el mensaje de fin, y al final *main* cierra el pipe y libera la memoria.

7. **Integración y compilación.** El *Makefile* lista los tres programas y los compila con *-Wall -Wextra -pthread*; el servidor se arma con *main.c* y *servidorISE.c*, y tanto él como el agente dependen de *servidorISE.h*, así que un cambio en el protocolo recompila ambos. Las llamadas principales tienen su comprobación de error con mensaje y código de salida 1: en el servidor *mkfifo*, *fork*, *waitpid*, *pthread_create* y *sem_init*, y en el agente *stat*, *open*, *fcntl* y *write*. Los recursos se devuelven con *liberar_slots* y *limpiar_recursos* en el servidor, y con *close* y *free* en el agente.
8. **Validación.** Se diseñaron pruebas funcionales que se ejecutaron a mano, se compilaron los tres programas con *make*, se generaron los CSV con *generadorCSV*, se levantó el servidor en una terminal y se lanzó un agente en cada una de las demás. Las pruebas de rendimiento se automatizaron con *medir_tiempos.c*, un programa aparte que *make* no compila y que usa *generadorCSV* para obtener los datos base; su funcionamiento se describe en el plan de pruebas.

**Arquitectura del servidor**

El servidor sigue la Figura 1 del enunciado y se divide en dos procesos. El proceso padre administra el pipe y espera al hijo; el hijo recibe las encuestas, calcula y muestra los resultados. Los hilos del hijo se resumen en la Tabla 2.

**Tabla 2**. *Procesos e hilos del servidor ISE*

| **Componente** | **Cantidad** | **Función** |
|---|---|---|
| Proceso padre | 1 | Crea el pipe, crea al hijo, espera su fin, borra el pipe e informa éxito o error. |
| Proceso hijo | 1 | Inicializa recursos, lanza los hilos y muestra el resumen final. |
| Hilo receptor | 1 | Único que lee el pipe (con *poll* de 500 ms); identifica el país y deposita cada encuesta en el buffer de su agente. |
| Hilo temporizador | 1 | Cuenta los segundos indicados con -t y ordena el cierre. |
| Hilo consumidor | Uno por agente (-a) | Toma las encuestas del buffer de su país, acumula sumas por sector y emite el reporte parcial al recibir el mensaje de fin. |

```
typedef enum { MSG_CONEXION, MSG_DATO, MSG_FIN } TipoMensaje;

typedef struct {
    TipoMensaje tipo;                   // tipo de mensaje
    char   pais[ISE_MAX_PAIS_TAM];      // codigo del pais emisor
    char   sector[ISE_MAX_SECTOR_LEN]; // nombre del sector
    int    id_sector;                   // sector de 0 a 4
    double sentimiento;                 // valor de 0.0 a 100.0
} MensajeEncuesta;
```

**Decisiones de sincronización**

- El receptor es el único lector, por lo que no hay competencia entre lectores y se conserva el orden de llegada.
- Cada mensaje mide menos que *PIPE_BUF*, por lo que el núcleo garantiza que un *write* no se mezcla con el de otro agente. Eso lo garantiza el sistema operativo y no los semáforos; el agente la comprueba en compilación con *_Static_assert*.
- El pipe abierto con O_RDWR en el servidor**.** Así el servidor mantiene un extremo de escritura abierto y *read* no devuelve fin de fichero cuando un agente se desconecta.
- Se tiene 2 semáforos por país, *espacios_libres* (iniciado en -b) e *items_disponibles* (iniciado en 0) controlan el buffer circular. Como cada buffer tiene un único productor (el receptor) y un único consumidor (el hilo de su agente), los índices no necesitan un *mutex* adicional.

- Se usa *sem_timedwait* con intervalos de un segundo, de modo que ningún hilo queda bloqueado para siempre al acabarse el tiempo o producirse un error.
- Usamos dos mutex, *mutex_asignacion* evita asignar dos espacios al mismo país; *mutex_tiempo* protege la bandera *tiempo_terminado*, que se consulta con *hay_que_cerrar* y se modifica con *marcar_cierre*.

**Distribución de tareas**

**Tabla 3**. *Distribución de tareas del grupo*

| **Integrante** | **Rol** | **Componentes** |
|---|---|---|
| Daniel Plazas | Servidor y arquitectura | servidorISE.c, servidorISE.h, main.c; redacción del documento. |
| Juan Camilo Maldonado | Generador y datos de prueba | generadorCSV.c; ficheros CSV de prueba, redacción del documento. |
| Samuel Zeudec | Agente y pruebas de integración | agentePais.c; pruebas con varios agentes, redacción del documento. |

## Plan de pruebas

El plan tiene dos partes: pruebas funcionales, que comprueban que el sistema hace lo que pide el enunciado, y pruebas de rendimiento, que miden cuánto tiempo toma recibir y procesar distintas cantidades de encuestas.

**Entorno y definiciones**

Las pruebas se ejecutaron en Linux, con los tres programas compilados con *make* (gcc con -Wall -Wextra -std=c11 -pthread, sin advertencias)  Las mediciones de rendimiento se hicieron en un equipo con 16 núcleos de CPU, los valores absolutos dependen del equipo, por lo que lo importante es la tendencia. Se usan las siguientes medidas:

- **Tiempo del agente:** desde que se lanza el agente hasta que termina (incluye leer el CSV, conectarse y enviar todas las encuestas).

- **Tiempo total:** desde que se lanza el agente hasta que el servidor imprime el reporte parcial de ese agente, es decir, envío más procesamiento.
- **Rendimiento:** encuestas divididas por el tiempo total, en encuestas por segundo.

El servidor permanece activo los -t segundos configurados (mínimo 30), por lo que ese tiempo no se mide: lo que se mide es lo que tarda en recibir y procesar las encuestas. Cada medición se repitió tres veces y se reporta el promedio. Para tener cantidades exactas (5000, 6000, etc.) se tomó una muestra aleatoria de N encuestas del CSV de Francia al 100 % (36640 encuestas), con semilla fija y con todos los sectores representados, y en cada ejecución se verificó que el total recibido por el servidor fuera igual a N.

Las mediciones las realiza el programa *medir_tiempos.c*, que se compila aparte con *gcc* (no lo compila *make*) y se ejecuta después de *make*, desde la carpeta del proyecto. Para cada medición, el programa lanza el servidor con *fork*, *dup2* y *execvp* (a través de *stdbuf -oL*, para que escriba línea a línea), con un pipe propio y un tiempo de 30 segundos, y espera a que imprima "Esperando conexiones". Luego lanza los agentes en paralelo y los espera con *waitpid*. El tiempo se mide con *clock_gettime* (*CLOCK_MONOTONIC*): el tiempo del agente termina cuando finaliza el último agente, y el tiempo total, cuando el servidor imprime el último "Reporte parcial". Si el total recibido de algún país no coincide con N, la medición se marca como inválida. Los resultados se muestran por pantalla y se guardan en *resultados_tiempos.csv* (Anexo C); son los de las Tablas 7, 8 y 9.

**Tabla 4**. *Diseño de los experimentos de rendimiento*

| N.° | Variable que cambia | Valores | Se mantiene fijo |
|---|---|---|---|
| 1 | Cantidad de encuestas | 1000, 5000, 6000, 10000, 20000 y 36640 | 1 agente, -b 10 |
| 2 | Tamaño del buffer (-b) | 1, 10, 100 y 1000 | 1 agente, 6000 encuestas |
| 3 | Cantidad de agentes a la vez | 1, 2 y 3 | 5000 encuestas por agente, -b 10 |

## Pruebas funcionales

La Tabla 5 resume los casos funcionales. En la prueba integral (caso 4) el servidor se lanzó con -b 10 -p ise_fifo -t 90 -a 3 y tres agentes (ES, FR e IT) en terminales distintas, uno de ellos con las banderas en otro orden. La salida completa se incluye en el Anexo B.

**Tabla 5**. *Casos de prueba funcionales y resultados obtenidos*

| N.° | Caso de prueba | Resultado esperado | Resultado obtenido |
|---|---|---|---|
| 1 | ./generadorCSV -p ES -e 100 | 6580 encuestas | Total 6580; CSV de 6581 líneas (con encabezado). |
| 2 | ./generadorCSV -p FR -e 75 | 27481 encuestas (ejemplo del enunciado) | 2888/3180/1253/2145/18015 = 27481; CSV de 27482 líneas. |
| 3 | ./generadorCSV -e 20 -p IT | 2620 encuestas, banderas en otro orden | Total 2620. |
| 4 | ./servidorISE -b 10 -p ise_fifo -t 90 -a 3 y tres agentes (ES, FR, IT) en terminales distintas | Cada total recibido coincide con el generador; ISE y recomendación; pipe eliminado; éxito del padre | Totales recibidos 6580, 27481 y 2620 (coinciden con el generador). ISE global 50,09 % y recomendación VENDER. Mensaje "Exito: el proceso ISE finalizo correctamente". ls ise_fifo falla: el pipe quedó borrado. |

| | | | |
|---|---|---|---|
| 5 | Servidor con -b 1 -a 2 y tres agentes | El tercer agente se rechaza con un aviso | Aviso de máximo de 2 agentes y rechazo de FR; totales de ES (6580) e IT (2620) correctos; código 0. |
| 6 | Servidor con -t 10 | Error por rango | "-t debe estar entre 30 y 120 segundos"; código 1. |
| 7 | Servidor con -a 5 | Error por rango | "-a debe estar entre 1 y 4"; código 1. |
| 8 | Agente sin servidor ni pipe activo | Error tras reintentar | Reintenta 10 veces y termina con el mensaje de que no hay servidor escuchando; código 1. |
| 9 | Agente con CSV sin encabezado válido | Error de formato | Informa que el fichero no tiene el encabezado esperado; código 1. |

**Tabla 6**. *Resultados de la prueba integral: encuestas recibidas y sentimiento promedio por sector*

| **Sector** | **Peso** | **ES (n / prom. %)** | **FR (n / prom. %)** | **IT (n / prom. %)** | **Global (prom. %)** |
|---|---|---|---|---|---|
| Industria | 40 % | 1760 / 50,31 | 2888 / 49,84 | 1088 / 50,15 | 50,04 |
| Servicios | 30 % | 1430 / 49,62 | 3180 / 50,20 | 620 / 50,79 | 50,11 |
| Consumidores | 20 % | 2020 / 49,96 | 1253 / 51,45 | 400 / 48,88 | 50,35 |
| Comercio | 5 % | 1080 / 48,96 | 2145 / 49,71 | 312 / 49,50 | 49,47 |
| Construcción | 5 % | 290 / 50,78 | 18015 / 49,99 | 200 / 51,29 | 50,02 |
| Total recibido | 100% | 6580 | 27481 | 2620 | 36681 |
| ISE global | | | | | 50,09 % |

**Pruebas de rendimiento: resultados**

La Tabla 7 muestra el tiempo que toma enviar y procesar distintas cantidades de encuestas con un solo agente, incluidas las de 5000 y 6000 encuestas.

**Tabla 7**. *Experimento 1: tiempo según la cantidad de encuestas (1 agente, -b 10)*

| Encuestas | Tiempo del agente (s) | Tiempo total (s) | Rendimiento (encuestas/s) |
|---|---|---|---|
| 1000 | 0,006 | 0,016 | 62500 |
| 5000 | 0,016 | 0,025 | 200000 |
| 6000 | 0,017 | 0,026 | 230769 |
| 10000 | 0,029 | 0,037 | 270270 |
| 20000 | 0,055 | 0,063 | 317460 |
| 36640 | 0,127 | 0,141 | 259858 |

**Tabla 8**. *Experimento 2: efecto del tamaño del buffer (1 agente, 6000 encuestas)*

| Buffer (-b) | Tiempo del agente (s) | Tiempo total (s) |
|---|---|---|
| 1 | 0,192 | 0,267 |
| 10 | 0,018 | 0,027 |
| 100 | 0,014 | 0,022 |
| 1000 | 0,013 | 0,022 |

**Tabla 9**. *Experimento 3: efecto de varios agentes a la vez (5000 encuestas por agente, -b 10)*

| Agentes | Encuestas en total | Tiempo total (s) | Rendimiento (encuestas/s) |
|---|---|---|---|
| 1 | 5000 | 0,021 | 238095 |
| 2 | 10000 | 0,043 | 232558 |
| 3 | 15000 | 0,056 | 267857 |

## Análisis

### Análisis 1: corrección funcional

En todas las pruebas funcionales los totales que recibe el servidor coinciden de manera exacta con los que imprime el generador (por ejemplo, 6580, 27481 y 2620 en la prueba integral), lo que indica que no se pierden ni se duplican encuestas, aunque tres agentes escriban a la vez en el mismo pipe. El ISE global de la prueba integral se obtiene como 0,40 × 50,04 + 0,30 × 50,11 + 0,20 × 50,35 + 0,05 × 49,47 + 0,05 × 50,02 = 50,09, y por ser más de 50 da la recomendación de vender. Los valores fuera de rango, la ausencia del servidor y los CSV con formato inválido producen errores controlados con código de salida 1, y con valores distintos a los de la prueba inicial el programa sigue funcionando.

### Análisis 2: tiempo según la cantidad de encuestas

Enviar y procesar 5000 o 6000 encuestas toma alrededor de 0,03 s (0,025 s y 0,026 s). Al pasar de 1000 a 36640 encuestas (unas 37 veces más), el tiempo total crece de 0,016 s a 0,141 s (unas 9 veces más), es decir, el tiempo aumenta de forma casi lineal, pero con un costo fijo. Con una recta entre los dos extremos se obtiene un costo cercano a 3,5 µs por encuesta más unos 12 ms fijos (arranque del agente, lectura del CSV y conexión al pipe). Ese costo fijo explica que el rendimiento suba de unas 62500 encuestas por segundo con 1000 encuestas a entre 260000 y

320000 con 20000 o más. Las pequeñas diferencias entre 5000 y 6000 (0,025 s y 0,026 s) están dentro de la variación normal de la medición.

**Análisis 3: efecto del tamaño del buffer**

Con -b 1 el tiempo total es de 0,267 s, unas diez veces mayor que con -b 10 (0,027 s). Con un solo espacio, el receptor solo puede dejar una encuesta y debe esperar a que el consumidor la saque antes de guardar la siguiente; los dos hilos se alternan en cada mensaje, con operaciones de semáforo y cambios de contexto en cada vuelta. Con buffers mayores el receptor puede adelantarse varias encuestas y las esperas son menos frecuentes. De 10 a 100 el tiempo baja de 0,027 s a 0,022 s y con 1000 se mantiene en 0,022 s, mientras que el propio agente tarda 0,013 s, lo que sugiere que a partir de cierto tamaño el buffer deja de ser el cuello de botella.

**Análisis 4: efecto de varios agentes simultáneos**

Al pasar de uno a tres agentes con 5000 encuestas cada uno, el tiempo total sube de 0,021 s a 0,056 s, es decir, unas 2,7 veces mientras se procesa el triple de datos. El rendimiento global pasa de unas 238000 a unas 268000 encuestas por segundo, una mejora pequeña (con dos agentes es prácticamente igual: 232558). Esto se debe a que cada agente tiene su propio buffer y su propio hilo consumidor, de modo que el procesamiento de los distintos países puede ocurrir en paralelo. La mejora no es proporcional porque todos los agentes comparten un único receptor y un único pipe, y porque el equipo de pruebas tiene un número limitado de 16 núcleos.

**Análisis 5: límites del diseño**

Las encuestas se reciben y procesan en menos de 0,2 s incluso con 36640 encuestas, pero el servidor permanece activo los -t segundos configurados (mínimo 30 s) y es el temporizador el que ordena el cierre. Esto implica que -t debe ser mayor que el tiempo que los agentes necesitan, al no ser asi, las encuestas que lleguen después del cierre no se van a procesar. Además, como el

pipe es de un solo sentido, un agente rechazado por superar el límite -a no se entera del rechazo y termina con el mensaje de éxito, aunque sus encuestas no sean procesadas.

## Conclusiones

1. **Corrección:** El sistema recibe y procesa todas las encuestas sin pérdidas ni duplicados con varios agentes, y calcula el ISE y la recomendación según los pesos del enunciado. La combinación de un único lector del pipe, mensajes de tamaño fijo menor que *PIPE_BUF* y semáforos por país resultó suficiente para evitar condiciones de carrera e interbloqueos.
2. **Tiempo y cantidad de encuestas:** El tiempo total crece de forma aproximadamente lineal con la cantidad de encuestas, con un costo cercano a 3,5 µs por encuesta más un costo fijo de unos 12 ms; enviar y procesar 5000 o 6000 encuestas toma alrededor de 0,03 s. El sistema escala bien hasta el máximo del enunciado (36640 encuestas en 0,14 s).
3. **Tamaño del buffer:** Un buffer de un solo registro multiplica por diez el tiempo, porque obliga a receptor y consumidor a alternarse en cada mensaje; con -b de 10 o más ese efecto casi desaparece. Por eso conviene usar un buffer de varios registros.
4. **Varios agentes:** Usar un buffer y un hilo consumidor por agente permite procesar los países en paralelo: el triple de datos tarda unas 2,7 veces más, una mejora pequeña. El límite lo ponen el único receptor, el único pipe y los núcleos disponibles.
5. **Límites del diseño.** El valor de -t debe ser mayor que lo que tardan los agentes, y el pipe de un solo sentido impide avisar a un agente rechazado. Así que se podría poner un segundo pipe de respuesta y un cierre que espere a los agentes activos.

**Resumen**

Este trabajo presenta un simulador del Indicador de Sentimiento Económico (ISE) desarrollado en C sobre Linux con procesos, hilos y semáforos POSIX. El sistema tiene tres programas: un generador de encuestas por país, agentes que envían las encuestas por un pipe nominal y un servidor que las recibe con un hilo receptor, un hilo temporizador y un hilo consumidor por agente, y que calcula el ISE como una media ponderada por sector y emite una recomendación de comprar, vender o mantenerse neutro. Se explican los conceptos usados, la metodología y las decisiones de sincronización. Las pruebas funcionales mostraron que los totales recibidos coinciden con los generados. Las pruebas de rendimiento mostraron que enviar y procesar 5000 o 6000 encuestas toma cerca de 0,03 s, que el tiempo crece de forma casi lineal con la cantidad de encuestas, que un buffer de un registro es unas diez veces más lento que uno de diez y que varios agentes simultáneos se procesan en paralelo con una mejora menos que proporcional. Se identificaron como límites que -t debe superar el tiempo de los agentes y que un agente rechazado no recibe aviso.

## Referencias

IEEE y The Open Group. (2018). *IEEE Std 1003.1-2017: Standard for information technology—Portable Operating System Interface (POSIX)*. The Open Group.

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Kerrisk, M. (s. f.). *pipe(7): Linux manual page*. Linux man-pages project. https://man7.org/linux/man-pages/man7/pipe.7.html

Pontificia Universidad Javeriana. (2026). *Proyecto: Indicador de Sentimiento Económico* [Enunciado del proyecto, Sistemas Operativos 2026-30]. Departamento de Ingeniería de Sistemas.

Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

## Anexo A. Compilación y ejecución

Para compilar y repetir la prueba integral, desde la carpeta del proyecto se ejecuta lo siguiente (cada agente en una terminal distinta):

```
$ make clean && make
$ ./generadorCSV -p ES -e 100
$ ./generadorCSV -p FR -e 75
$ ./generadorCSV -e 20 -p IT

Terminal 1:
$ ./servidorISE -b 10 -p ise_fifo -t 90 -a 3 | tee salida_servidor.txt

Terminales 2 a 4:
$ ./agentePais -n encuestas_ES.csv -p ise_fifo
$ ./agentePais -n encuestas_FR.csv -p ise_fifo
$ ./agentePais -p ise_fifo -n encuestas_IT.csv

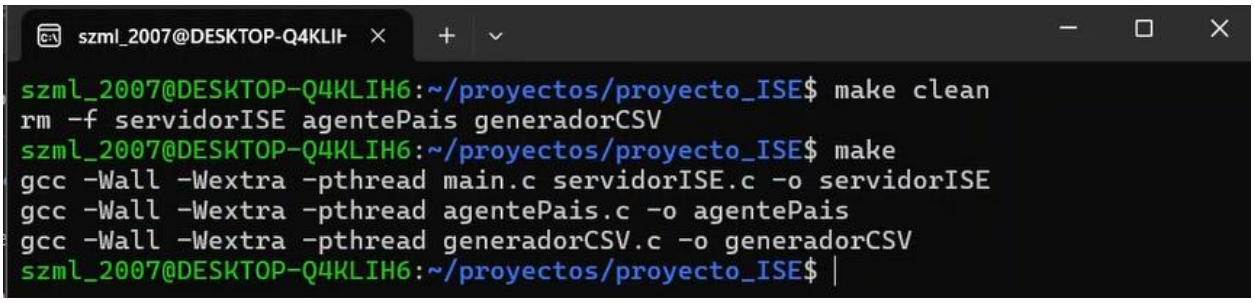
Verificacion:
$ grep "Total" salida_servidor.txt
$ ls ise_fifo        # debe fallar: el pipe se elimina al terminar

Mediciones de tiempo (opcional):
$ ./medir_tiempos.sh
```

## Anexo B. Salida de la prueba integral y pruebas funcionales

A continuación, se indican los comandos escritos y, debajo de cada uno, lo que mostró el programa.

### Compilación




### Generación de los ficheros CSV

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./generadorCSV -p ES -e 100
Generando fichero encuestas_ES.csv para pais ES con 100% de encuestas
Maximos por sector:
  Industria     : 1760/1760 encuestas (100% del maximo)
  Servicios     : 1430/1430 encuestas (100% del maximo)
  Consumidores : 2020/2020 encuestas (100% del maximo)
  Comercio      : 1080/1080 encuestas (100% del maximo)
  Construcci n : 290/290 encuestas (100% del maximo)
Total encuestas: 6580
Fichero CSV generado
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./generadorCSV -p FR -e 75
Generando fichero encuestas_FR.csv para pais FR con 75% de encuestas
Maximos por sector:
  Industria     : 2888/3850 encuestas (75% del maximo)
  Servicios     : 3180/4240 encuestas (75% del maximo)
  Consumidores : 1253/1670 encuestas (75% del maximo)
  Comercio      : 2145/2860 encuestas (75% del maximo)
  Construcci n : 18015/24020 encuestas (75% del maximo)
Total encuestas: 27481
Fichero CSV generado
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./generadorCSV -e 20 -p IT
Generando fichero encuestas_IT.csv para pais IT con 20% de encuestas
Maximos por sector:
  Industria     : 1088/5440 encuestas (20% del maximo)
  Servicios     : 620/3100 encuestas (20% del maximo)
  Consumidores : 400/2000 encuestas (20% del maximo)
  Comercio      : 312/1560 encuestas (20% del maximo)
  Construcci n : 200/1000 encuestas (20% del maximo)
Total encuestas: 2620
Fichero CSV generado
```

### Agentes lectura

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./agentePais -n encuestas_ES.csv -p ise_fifo
==========================================
      AGENTE LECTURA - ENVIO DE ENCUESTAS
==========================================
Pais: ES
Pipe: ise_fifo
Leidas 6580 encuestas.
Conectando al proceso principal...
Conectado.
Enviando...
  500/6580
  1000/6580
  1500/6580
  2000/6580
  2500/6580
  3000/6580
  3500/6580
  4000/6580
  4500/6580
  5000/6580
  5500/6580
  6000/6580
  6500/6580
Enviadas 6580/6580 encuestas.
Mensaje de finalizacion enviado.
Envio ES finalizado satisfactoriamente!!.
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$
```

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./agentePais -n encuestas_FR.csv -p ise_fifo
==============================================
      AGENTE LECTURA - ENVIO DE ENCUESTAS
==============================================
Pais: FR
Pipe: ise_fifo
Leidas 27481 encuestas.
Conectando al proceso principal...
Conectado.
Enviando...
  500/27481
  1000/27481
  1500/27481
  2000/27481
  2500/27481
  3000/27481
  3500/27481
  4000/27481
  4500/27481
  5000/27481
  5500/27481
  6000/27481
  6500/27481
  7000/27481
  7500/27481
  8000/27481
  8500/27481
  9000/27481
  9500/27481
  10000/27481
  10500/27481
  11000/27481
  11500/27481
  12000/27481
  12500/27481
  13000/27481
  13500/27481
  14000/27481
  14500/27481
  15000/27481
  15500/27481
  16000/27481
  16500/27481
  17000/27481
  17500/27481
  18000/27481
  18500/27481
  19000/27481
  19500/27481
  20000/27481
  20500/27481
```

```
  21000/27481
  21500/27481
  22000/27481
  22500/27481
  23000/27481
  23500/27481
  24000/27481
  24500/27481
  25000/27481
  25500/27481
  26000/27481
  26500/27481
  27000/27481
Enviadas 27481/27481 encuestas.
Mensaje de finalizacion enviado.
Envio FR finalizado satisfactoriamente!!.
```

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./agentePais -p ise_fifo -n encuestas_IT.csv
================================================
      AGENTE LECTURA - ENVIO DE ENCUESTAS
================================================
Pais: IT
Pipe: ise_fifo
Leidas 2620 encuestas.
Conectando al proceso principal...
Conectado.
Enviando...
  500/2620
  1000/2620
  1500/2620
  2000/2620
  2500/2620
Enviadas 2620/2620 encuestas.
Mensaje de finalizacion enviado.
Envio IT finalizado satisfactoriamente!!.
```

## Servidor ISE

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./servidorISE -b 10 -p ise_fifo -t 90 -a 3 | tee s
alida_servidor.txt
Control Proceso Principal ISE iniciado.
 Pipe nominal : ise_fifo
 Tiempo limite: 90 segundos
 Agentes max  : 3
 Tam. buffer  : 10 registros por agente
Esperando conexiones de Agentes...
Conexion establecida con Agente: ES

[Reporte parcial - Agente ES]
 Industria    :   1760 encuestas, sentimiento promedio 50.31%
 Servicios    :   1430 encuestas, sentimiento promedio 49.62%
 Consumidores :   2020 encuestas, sentimiento promedio 49.96%
 Comercio     :   1080 encuestas, sentimiento promedio 48.96%
 Construccion :    290 encuestas, sentimiento promedio 50.78%
 Total ES: 6580 encuestas recibidas.
Conexion establecida con Agente: FR

[Reporte parcial - Agente FR]
 Industria    :   2888 encuestas, sentimiento promedio 49.84%
 Servicios    :   3180 encuestas, sentimiento promedio 50.20%
 Consumidores :   1253 encuestas, sentimiento promedio 51.45%
 Comercio     :   2145 encuestas, sentimiento promedio 49.71%
 Construccion :  18015 encuestas, sentimiento promedio 49.99%
 Total FR: 27481 encuestas recibidas.
Conexion establecida con Agente: IT

[Reporte parcial - Agente IT]
 Industria    :   1088 encuestas, sentimiento promedio 50.15%
 Servicios    :    620 encuestas, sentimiento promedio 50.79%
 Consumidores :    400 encuestas, sentimiento promedio 48.88%
 Comercio     :    312 encuestas, sentimiento promedio 49.50%
 Construccion :    200 encuestas, sentimiento promedio 51.29%
 Total IT: 2620 encuestas recibidas.

Tiempo del servidor finalizado. Cerrando recepcion...

 ------------------------ RESUMEN DEL ISE ----------------------------------
 Industria     (peso   40%): promedio 50.04%
 Servicios     (peso   30%): promedio 50.11%
 Consumidores  (peso   20%): promedio 50.35%
 Comercio      (peso    5%): promedio 49.47%
 Construccion  (peso    5%): promedio 50.02%

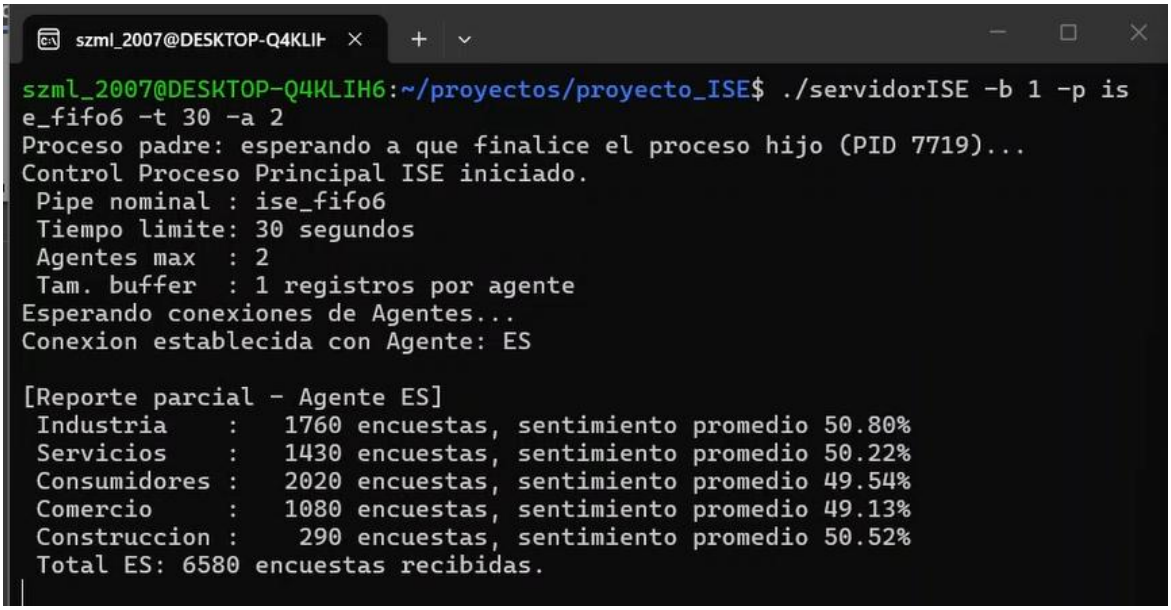
 Indicador ISE global: 50.09%
 Recomendacion: VENDER
============================================================
Proceso padre: esperando a que finalice el proceso hijo (PID 25930)...
Exito: el proceso ISE finalizo correctamente.
```

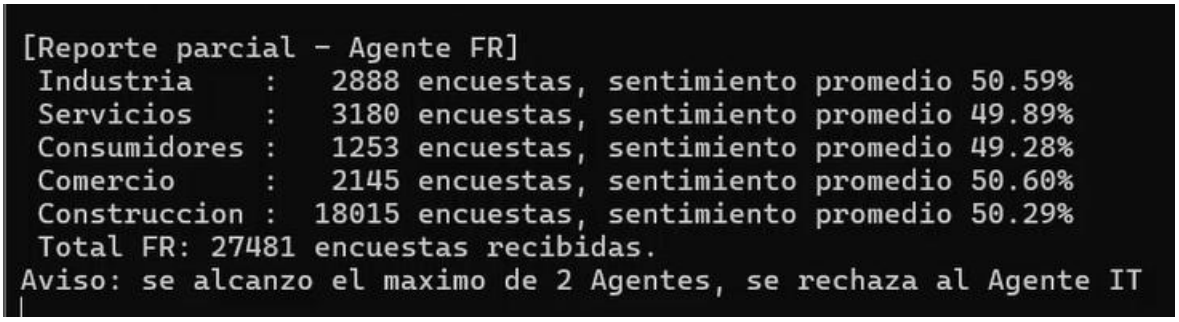
## Verificación

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ grep "Total" salida_servidor.txt
 Total ES: 6580 encuestas recibidas.
 Total FR: 27481 encuestas recibidas.
 Total IT: 2620 encuestas recibidas.
```

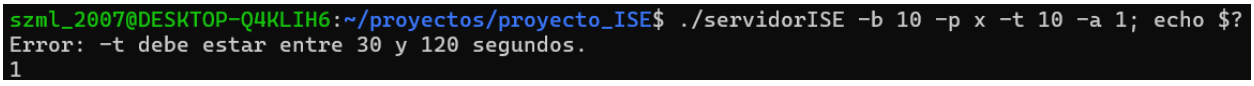
```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ls ise_fifo
ls: cannot access 'ise_fifo': No such file or directory
```

**Servidor con -b 1 -a 2 y tres agentes**

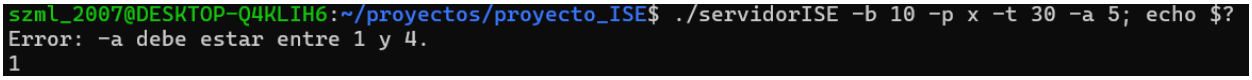



**Servidor con -t 10**




**Servidor con -a 5**



### Agente sin servidor o pipe activo

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./agentePais -n encuestas_ES.csv -p noexiste; echo $?
==============================================
      AGENTE LECTURA - ENVIO DE ENCUESTAS
==============================================
Pais: ES
Pipe: noexiste
Leidas 66 encuestas.
Conectando al proceso principal...
Error: no se puede acceder al pipe 'noexiste': No such file or directory
1
```

### Agente con CSV sin encabezado válido

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./agentePais -n csv_sin_encabezado.csv -p x;echo $?
Error: no se pudo abrir 'csv_sin_encabezado.csv': No such file or directory
1
```

## Anexo C. Salida del script de mediciones

Salida de *medir_tiempos.c*, con promedios de tres repeticiones por medición.

```
szml_2007@DESKTOP-Q4KLIH6:~/proyectos/proyecto_ISE$ ./medir_tiempos.c | tee salida_mediciones.txt
Generando CSV base (ES, FR e IT al 100 %)...

Experimento 1: tiempo segun la cantidad de encuestas (1 agente, -b 10)
Encuestas    Tiempo agente (s)  Tiempo total (s) Rendimiento (enc/s)    Estado
1000         0.006              0.016            62500                 OK
5000         0.016              0.025            200000                OK
6000         0.017              0.026            230769                OK
10000        0.029              0.037            270270                OK
20000        0.055              0.063            317460                OK
36640        0.127              0.141            259858                OK

Experimento 2: efecto del tamano del buffer (1 agente, 6000 encuestas)
Buffer -b    Tiempo agente (s)  Tiempo total (s) Estado
1            0.192              0.267            OK
10           0.018              0.027            OK
100          0.014              0.022            OK
1000         0.013              0.022            OK

Experimento 3: efecto de varios agentes a la vez (5000 encuestas por agente, -b 10)
Agentes    Encuestas totales    Tiempo total (s) Rendimiento (enc/s)    Estado
1          5000                 0.021            238095                OK
2          10000                0.043            232558                OK
3          15000                0.056            267857                OK
```